Informe de Seguridad del Sistema Académico *Guia_3*: Usuarios, Privilegios, Roles y Auditoría en MySQL 8

Juan Manuel Gámez Nieto, Esteban Salvador Guzmán y Joan Sebastián Romaña Tavera

Programa de Ingeniería de Software, Universidad Manuela Beltrán

Teoría de Bases de Datos: Guía de Laboratorio N.º 4, Actividad 5

Docente: _____

5 de octubre de 2026

Tabla de contenido

Resumen 4

Introducción 5

Marco conceptual 5

- Principios de seguridad de la información 5
- Autenticación y autorización 6
- Modelos de control de acceso 6
- Auditoría de bases de datos 7
- Inyección SQL 7
- Marco normativo colombiano 7

Metodología y entorno de trabajo 8

Objetivo y alcance 8

Arquitectura de seguridad por capas 9

Mapa de seguridad 10

- Inventario de usuarios 10
- Inventario de roles 10
- Diagrama de usuarios, roles y objetos 11
- Matriz de privilegios efectivos 12
- Privilegio directo frente a privilegio heredado 15
- Mecanismos de auditoría 17
- Control en la capa de aplicación 20

Evidencias del mapa de seguridad 21

Validación técnica del mapa 22

Análisis de riesgos 23

Recomendaciones para producción 25

- Autenticación 25
- Autorización 26
- Auditoría 28
- Aplicación 29
- Operación e infraestructura 29

Hoja de ruta priorizada 30
Correspondencia con controles NIST SP 800-53 30
Respuestas a las preguntas orientadoras 31
Conclusiones 32
Referencias 33
Anexo A. Script `05_Actividad5_Mapa_Seguridad.sql` 34
Anexo B. Salida de referencia del script 05 37
Anexo C. Triggers de auditoría corregidos 40
Anexo D. Validación de entradas en la aplicación 42

Resumen

Este informe consolida la Guía de Laboratorio N.º 4 de Teoría de Bases de Datos. Documenta el mapa de seguridad del sistema académico `Guia_3`, construido en MySQL 8 durante las prácticas 1 a 3. El sistema tiene seis tablas de negocio y una tabla de auditoría. La seguridad se organizó en cinco capas: autenticación con cuentas atadas a `localhost`; autorización discrecional por tabla; control de acceso basado en roles (`rol_admin`, `rol_consultor` y `rol_operador`); consultas parametrizadas en la aplicación Python; y auditoría mediante triggers `AFTER UPDATE` y `AFTER DELETE` que registran en `auditoria_log` quién hizo el cambio, cuándo lo hizo y cuál era el valor anterior. Las pruebas confirmaron con `ERROR 1142` todas las operaciones prohibidas para cada perfil. El análisis identificó once hallazgos. Dos ya se corrigieron: el registro del usuario responsable con `USER()` en lugar de `CURRENT_USER()` y la validación de entradas. Entre los abiertos destaca la falta de separación de funciones, porque el administrador puede borrar el log que lo vigila. Se proponen diecinueve recomendaciones para producción, ordenadas por capa y por prioridad, y se relacionan con los controles de NIST SP 800-53 y con la Ley 1581 de 2012. Las medidas clave se validaron en un servidor MySQL 8.0 de prueba.

Palabras clave: seguridad de bases de datos, mínimo privilegio, RBAC, auditoría, triggers, inyección SQL, MySQL

Informe de Seguridad del Sistema Académico Guia_3

Una base de datos académica guarda información que debe protegerse: documentos de identidad, teléfonos, direcciones, fechas de nacimiento, notas y valores de matrícula. Si una persona sin autorización puede leer, alterar o borrar esos datos, el daño no es solo técnico. También es legal, porque se trata de datos personales protegidos en Colombia, y es institucional, porque una nota modificada o un pago borrado afecta directamente a los estudiantes.

En esta guía el equipo asumió el rol de administrador de bases de datos (DBA) del sistema `Guia_3`. Las actividades 1 y 2 crearon usuarios, privilegios y roles. La actividad 3 mostró cómo se explota una inyección SQL y cómo se evita. La actividad 4 construyó un rastro de auditoría con triggers. Este informe corresponde a la actividad 5. Reúne todo lo anterior en un mapa de seguridad que indica qué cuentas existen, qué puede hacer cada una y qué mecanismos registran los cambios. Luego analiza los riesgos que siguen abiertos y propone las medidas necesarias para llevar el sistema a producción.

El documento comienza con el marco conceptual y la metodología. Después presenta el mapa de seguridad, la validación técnica, el análisis de riesgos y las recomendaciones. Termina con las respuestas a las preguntas orientadoras de la guía y las conclusiones. Los anexos contienen el script de consulta del mapa, su salida de referencia y el código corregido de los triggers y de la validación de entradas.

Marco conceptual

Principios de seguridad de la información

La seguridad de una base de datos busca preservar tres propiedades, conocidas como la tríada CIA. La *confidencialidad* exige que solo las personas autorizadas lean la información. La *integridad* exige que solo las personas autorizadas la modifiquen, y por los medios previstos. La *disponibilidad* exige que la información esté accesible cuando se necesite (Elmasri & Navathe, 2016). Cada control de este informe protege al menos una de ellas. Por ejemplo, el `SELECT` por tabla protege la confidencialidad, la ausencia de `DELETE` para el operador protege la integridad y las copias de seguridad protegen la disponibilidad.

Dos principios guían el diseño. El *mínimo privilegio* establece que cada cuenta tenga solo los permisos que su función necesita (Joint Task Force, 2020, control AC-6). La *separación de funciones* establece que ninguna persona pueda ejecutar y ocultar por sí sola una acción sensible; por ejemplo, quien administra los datos no debería poder borrar el registro que lo vigila (Joint Task Force, 2020, control AC-5).

Autenticación y autorización

La *autenticación* responde a la pregunta "¿quién eres?". En MySQL una cuenta es el par `usuario@host`, protegido con una contraseña, de modo que el servidor verifica a la vez la identidad y el origen de la conexión. La *autorización* responde a la pregunta "¿qué puedes hacer?". Una vez autenticado el usuario, el servidor revisa sus privilegios en las tablas de permisos (`mysql.user`, `mysql.db`, `mysql.tables_priv` y `mysql.columns_priv`) antes de ejecutar cada sentencia (Oracle, 2024b). Si falta el privilegio, la sentencia se rechaza con `ERROR 1142` (*command denied*).

Modelos de control de acceso

La literatura distingue tres modelos principales (Elmasri & Navathe, 2016; Silberschatz et al., 2019). La Tabla 1 los compara y señala cuáles se usan en este sistema.

Tabla 1

Modelos de control de acceso y su uso en Guia_3

Modelo	**Idea central**	**Mecanismo en MySQL**	**Uso en Guia_3**
DAC (discrecional)	El dueño de un objeto decide a quién otorga permisos y puede delegarlos	`GRANT` / `REVOKE`, `WITH GRANT OPTION`	Actividad 1: privilegios directos por usuario y por tabla
RBAC (basado en roles)	Los permisos se asignan a un cargo (rol) y los usuarios heredan los del rol	`CREATE ROLE`, `GRANT rol TO usuario`, `SET DEFAULT ROLE`	Actividad 2: `rol_admin`, `rol_consultor`, `rol_operador`
MAC (obligatorio)	Etiquetas de clasificación (secreto, confidencial…) que el usuario no puede cambiar	No nativo en MySQL Community	No aplica; las vistas con enmascaramiento (R8) cubren parte de la necesidad

Auditoría de bases de datos

La auditoría registra los eventos relevantes para poder reconstruir después quién hizo qué, cuándo y sobre qué dato. Un registro útil debe responder esas preguntas y además estar protegido contra su modificación (Joint Task Force, 2020, controles AU-3 y AU-9). En MySQL Community la forma más directa de auditar cambios de datos es con *triggers*: procedimientos que el motor ejecuta automáticamente antes o después de un `INSERT`, `UPDATE` o `DELETE` sobre una tabla. Dentro del trigger los valores previos están en `OLD.*` y los nuevos en `NEW.*` (Oracle, 2024e).

Inyección SQL

La inyección SQL ocurre cuando una aplicación construye una consulta uniendo texto con datos que escribe el usuario. Si la entrada contiene fragmentos de SQL, como `' OR '1'='1` o un comentario `--`, el motor los interpreta como parte de la instrucción y cambia su lógica. OWASP la clasifica dentro de la categoría A03:2021 (*Injection*) de su Top 10 (OWASP Foundation, 2021). La defensa principal es la consulta parametrizada, en la que la estructura de la sentencia se fija antes de recibir los datos. Se complementa con validación de entradas y con cuentas de mínimo privilegio (OWASP Foundation, 2023).

Marco normativo colombiano

La Ley Estatutaria 1581 de 2012 regula el tratamiento de datos personales en Colombia y el Decreto 1377 de 2013 la reglamenta parcialmente. Para este sistema son especialmente relevantes tres principios del artículo 4 de la ley. El principio de *acceso y circulación restringida* exige que solo accedan a los datos las personas autorizadas. El de *seguridad* exige medidas técnicas que eviten la adulteración, pérdida, consulta o uso no autorizado. El de *confidencialidad* obliga a reservar la información (Congreso de la República de Colombia, 2012). Los privilegios por tabla, los roles, la auditoría y el enmascaramiento propuesto (R8) son medidas técnicas que responden a estos principios.

Metodología y entorno de trabajo

El trabajo se realizó sobre MySQL 8.x en entorno local (`localhost:3306`). Los scripts se editaron y ejecutaron desde Visual Studio Code con la extensión SQLTools y desde MySQL Workbench. La aplicación de la actividad 3 se escribió en Python 3 con el conector oficial `mysql-connector-python`. El orden de ejecución fue el siguiente: `01_Tablas.sql` y `02_Datos.sql` (estructura y datos), la tarea previa sobre `GRANT`/`REVOKE`, la actividad 1 (usuarios y privilegios directos), la actividad 2 (roles), la actividad 3 (inyección SQL), la actividad 4 (triggers de auditoría) y finalmente el script de la actividad 5, que solo consulta el catálogo.

La base de datos contiene 5 programas, 5 estudiantes, 5 profesores, 5 materias, 5 grupos y 13 matrículas. Para cada usuario se abrió una sesión independiente y se ejecutó su bloque de pruebas. Las operaciones permitidas debían completarse y las prohibidas debían fallar con `ERROR 1142`. Por último, para verificar este informe, todos los scripts se volvieron a ejecutar en un servidor MySQL 8.0.46 de prueba. Allí se comprobaron la matriz de privilegios, el funcionamiento de los triggers y la sintaxis de las recomendaciones de producción (ver la sección *Validación técnica del mapa*).

> **Nota sobre el nombre del esquema.** El sistema de referencia es `Guia_3`. Algunas versiones de los scripts de apoyo (actividades 3 y 4) apuntan a una copia de trabajo llamada `Guia_4`, que tiene la misma estructura. Los resultados y las conclusiones son los mismos en ambos esquemas.

Objetivo y alcance

El objetivo es documentar el mapa de seguridad del sistema académico `Guia_3` tal como quedó tras las actividades 1 a 4: qué cuentas existen, qué roles agrupan sus permisos, qué puede hacer cada cuenta sobre cada tabla y qué mecanismos registran los cambios. También se analizan los riesgos que siguen abiertos y se proponen las medidas necesarias para llevar el sistema a producción. La Tabla 2 delimita el alcance.

Tabla 2

Alcance del informe

Incluido	Fuera de alcance
6 tablas de negocio: `Programa`, `Estudiante`, `Profesor`, `Materia`, `Grupo`, `Matricula`	Seguridad del sistema operativo y del hardware
Tabla de auditoría `auditoria_log`	Cuentas internas de MySQL (`mysql.sys`, `mysql.session`, `mysql.infoschema`)
4 usuarios, 3 roles y 2 triggers	Aplicaciones distintas a los scripts de la actividad 3
Código de acceso a datos (Python y MySQL Connector)	Seguridad física y de la red institucional

El principio rector es el *mínimo privilegio*: cada cuenta tiene solo los permisos que su función necesita (Elmasri & Navathe, 2016; Joint Task Force, 2020, control AC-6).

Arquitectura de seguridad por capas

La protección del sistema no depende de un solo control. Se organiza en capas, de modo que si una falla, la siguiente limita el daño. Este enfoque se conoce como *defensa en profundidad*. La Tabla 3 resume las cinco capas implementadas.

Tabla 3

Capas de seguridad implementadas en Guia_3

#	Capa	Pregunta que responde	Control implementado	Actividad
1	**Autenticación**	¿Quién eres y desde dónde te conectas?	`CREATE USER 'usuario'@'localhost' IDENTIFIED BY ...` (cuenta, host y contraseña)	1
2	**Autorización (DAC)**	¿Qué puedes hacer y sobre qué objeto?	`GRANT` / `REVOKE` por tabla y por operación	1
3	**Autorización (RBAC)**	¿Qué permisos corresponden a tu cargo?	Roles `rol_admin`, `rol_consultor`, `rol_operador`	2
4	**Aplicación**	¿Puede un formulario inyectar SQL?	Consultas parametrizadas (*prepared statements*) y validación de entradas	3
5	**Auditoría**	¿Quién cambió qué, cuándo y qué había antes?	Triggers `AFTER UPDATE` / `AFTER DELETE` que escriben en `auditoria_log`	4

Mapa de seguridad

Inventario de usuarios

El sistema tiene cuatro cuentas de usuario, cada una con una función distinta. La Tabla 4 indica para cada cuenta el host permitido, su función, el origen de sus permisos y su alcance.

Tabla 4

Inventario de usuarios del sistema

Usuario	Host	Perfil / función	Origen de permisos	Alcance
`admin_bd`	`localhost`	DBA: administra el esquema y delega permisos	Heredado de `rol_admin`	`ALL PRIVILEGES` sobre `Guia_3.*` con `GRANT OPTION`
`analista`	`localhost`	Consulta y genera reportes; no modifica datos	Heredado de `rol_consultor`	`SELECT` sobre las 6 tablas de negocio
`operador`	`localhost`	Registra y corrige datos del día a día; no borra	Heredado de `rol_operador`	`SELECT, INSERT, UPDATE` sobre las 6 tablas de negocio
`auditor`	`localhost`	Revisa el rastro de cambios; independiente de la operación	Directo (sin rol)	`SELECT` sobre `auditoria_log` únicamente

Nota. Las contraseñas de laboratorio no se reproducen en este informe (ver hallazgo H1).

En MySQL una cuenta es el par *usuario* + *host*. Restringir el host a `localhost` significa que, aunque alguien robe la contraseña, no puede usarla desde otra máquina de la red. Es la primera barrera de autenticación. Una cuenta `'analista'@'%'`, en cambio, aceptaría conexiones desde cualquier equipo.

Inventario de roles

Los tres roles de la Tabla 5 agrupan los permisos por cargo. Cada rol se asignó a un usuario y se dejó activo por defecto.

Tabla 5

Inventario de roles

Rol	Privilegios que agrupa	Asignado a	Activo por defecto
`rol_admin`	`ALL PRIVILEGES ON Guia_3.* WITH GRANT OPTION`	`admin_bd@localhost`	Sí
`rol_consultor`	`SELECT` sobre `Programa`, `Estudiante`, `Profesor`, `Materia`, `Grupo`, `Matricula`	`analista@localhost`	Sí
`rol_operador`	`SELECT, INSERT, UPDATE` sobre las mismas 6 tablas	`operador@localhost`	Sí

En MySQL 8 un rol se guarda internamente como una cuenta **bloqueada y sin contraseña** (`account_locked = 'Y'`). Por eso aparece como `rol_consultor@%`. El `%` no es un riesgo, porque nadie puede iniciar sesión con un rol (Oracle, 2024a).

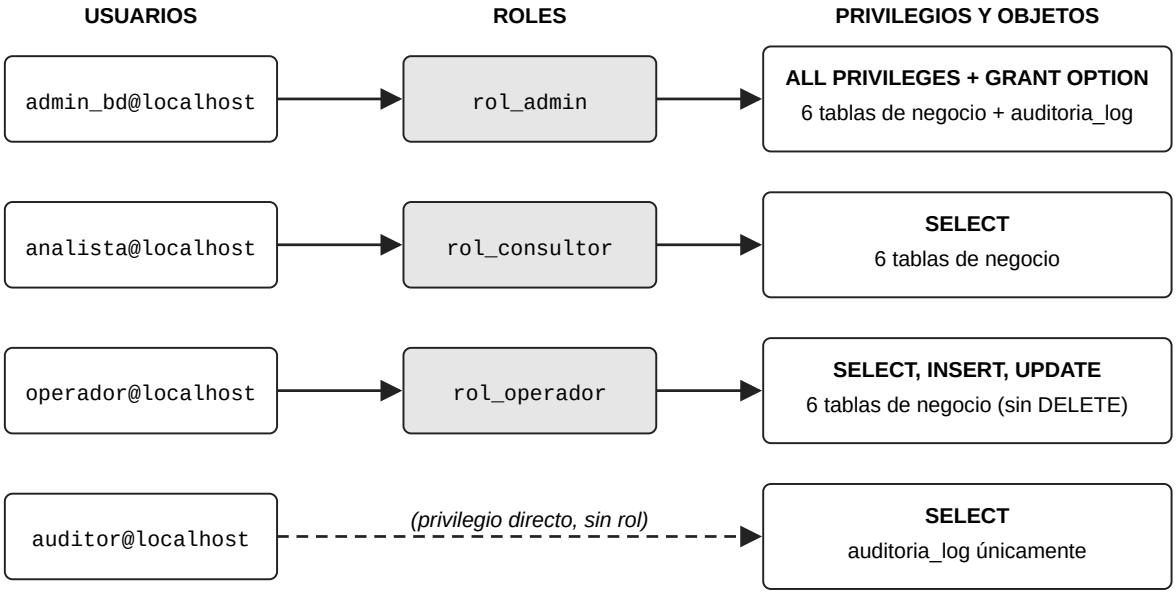
La instrucción `SET DEFAULT ROLE` hace que el rol quede activo apenas el usuario se conecta. Sin ella el usuario tendría el rol asignado pero inactivo, y sus consultas fallarían con `ERROR 1142` hasta ejecutar manualmente `SET ROLE`. Una alternativa global es la variable `activate_all_roles_on_login`, que en este sistema se dejó en `OFF` para controlar la activación cuenta por cuenta.

Diagrama de usuarios, roles y objetos

La Figura 1 muestra cómo fluyen los privilegios desde los usuarios, a través de los roles, hasta los objetos del esquema.

Figura 1

Relación entre usuarios, roles y objetos del esquema Guia_3



Nota. Las flechas continuas representan privilegios heredados por rol; la flecha discontinua, un privilegio otorgado directamente. Los roles (en gris) son cuentas bloqueadas que no permiten iniciar sesión.

Matriz de privilegios efectivos

La Tabla 6 cruza cada tabla del esquema con cada cuenta y muestra los privilegios efectivos, es decir, los que la cuenta puede usar una vez activado su rol.

Tabla 6

Matriz de privilegios efectivos por tabla y por cuenta

Tabla	admin_bd	analista	operador	auditor
Programa	ALL + G	S	S I U	—
Estudiante	ALL + G	S	S I U	—
Profesor	ALL + G	S	S I U	—
Materia	ALL + G	S	S I U	—
Grupo	ALL + G	S	S I U	—
Matricula	ALL + G	S	S I U	—
auditoria_log	ALL + G	—	—	**S**

Nota. S = SELECT; I = INSERT; U = UPDATE; ALL = todos los privilegios del esquema, incluidos los de definición (CREATE, ALTER, DROP, TRIGGER, entre otros); G = puede delegar (GRANT OPTION); — = sin acceso.

La matriz refleja cuatro decisiones de diseño:

1. **Los permisos de analista y operador se otorgan tabla por tabla, nunca con Guia_3.*.** Un comodín de esquema incluiría auditoria_log sin que nadie lo notara, y también cualquier tabla que se cree en el futuro. Así el log queda fuera del alcance de quien opera los datos.
2. **operador no tiene DELETE.** Un error de digitación puede corregirse con UPDATE, pero un borrado es irreversible y puede romper la integridad referencial (Matricula depende de Estudiante y de Grupo). Borrar queda reservado al DBA.
3. **auditor no ve los datos de negocio.** Solo necesita el log para hacer su trabajo. Es el principio de *necesidad de saber*.
4. **operador dispara los triggers sin tener INSERT sobre auditoria_log.** El trigger se ejecuta con los permisos de su DEFINER, no con los del usuario que hizo el cambio (Oracle, 2024c). Por eso el registro de auditoría ocurre aunque el operador no pueda escribir en el log, y por la misma razón tampoco puede falsificarlo.

Las pruebas de acceso de las actividades 1 y 2 confirmaron la matriz. La Tabla 7 resume lo que cada cuenta puede y no puede hacer.

Tabla 7

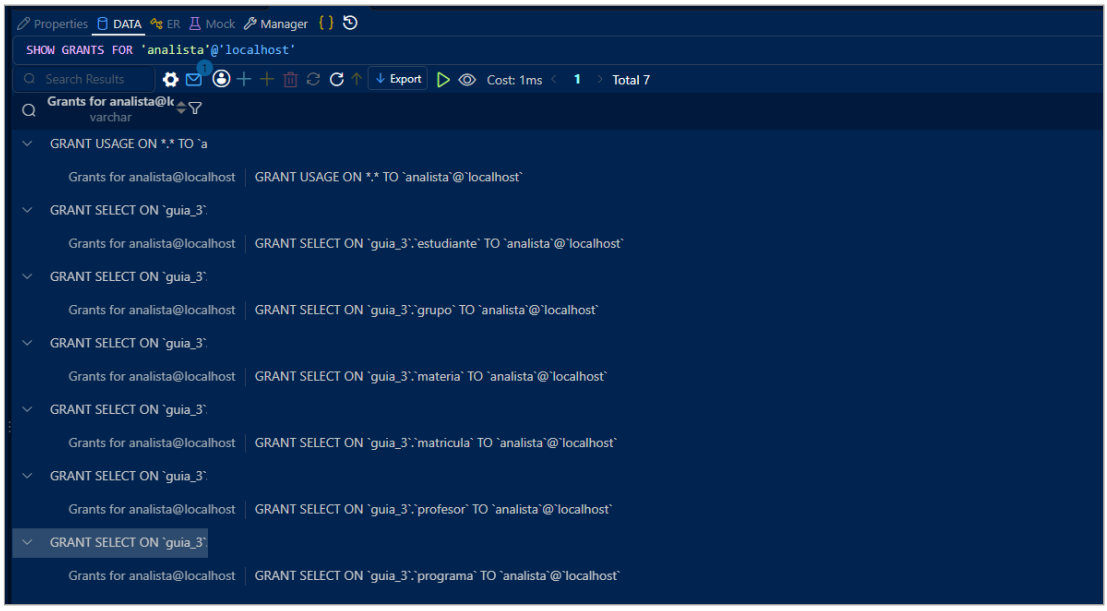
Resultado de las pruebas de acceso (actividades 1 y 2)

Usuario	Puede	No puede (error obtenido)
`admin_bd`	Consultar, insertar, actualizar y borrar en todas las tablas; otorgar y revocar permisos	—
`analista`	`SELECT` en las 6 tablas de negocio	`INSERT`, `UPDATE` y `DELETE` en cualquier tabla; `SELECT` sobre `auditoria_log` → `ERROR 1142`
`operador`	`SELECT`, `INSERT` y `UPDATE` en las 6 tablas	`DELETE` en cualquier tabla; `SELECT` sobre `auditoria_log` → `ERROR 1142`
`auditor`	`SELECT` sobre `auditoria_log`	`SELECT` sobre cualquier tabla de negocio; `INSERT` en el log → `ERROR 1142`

Las Figuras 2 a 4 muestran la salida de `SHOW GRANTS` al final de la actividad 1, cuando los privilegios todavía eran directos.

Figura 2

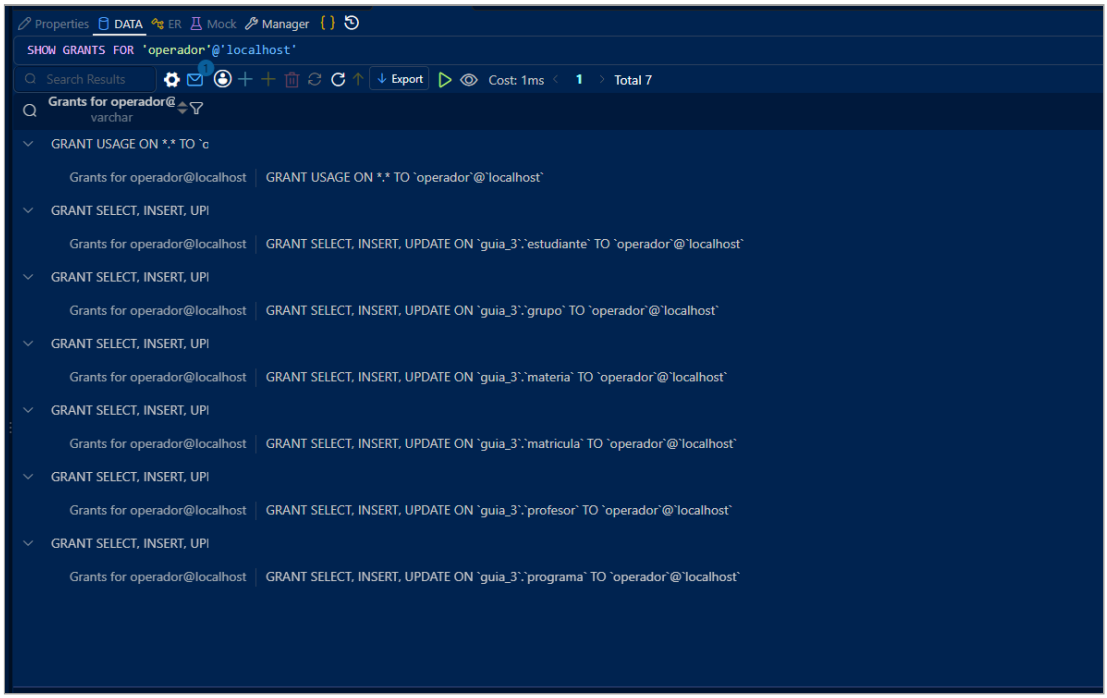
Privilegios directos de analista en la actividad 1



Nota. Salida de `SHOW GRANTS FOR 'analista'@'localhost'`: un `SELECT` por cada una de las 6 tablas de negocio y ningún permiso sobre `auditoria_log`.

Figura 3

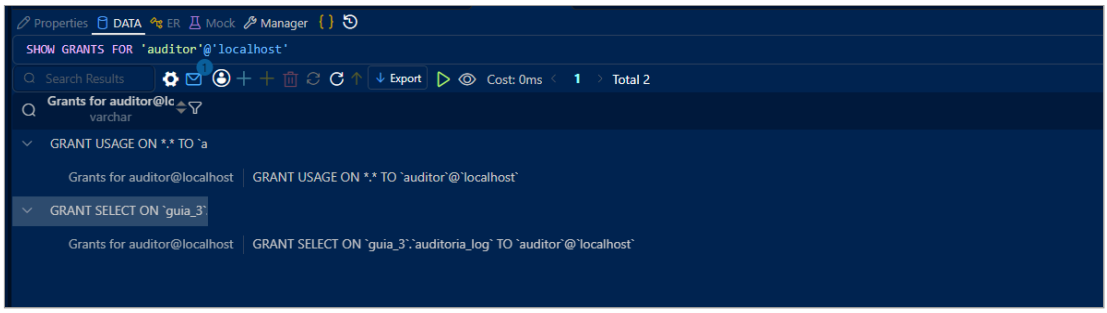
Privilegios directos de operador en la actividad 1



Nota. `SELECT, INSERT, UPDATE` tabla por tabla, sin `DELETE`.

Figura 4

Privilegio directo de auditor en la actividad 1



Nota. El único privilegio de `auditor` es `SELECT` sobre `guia_3.auditoria_log`. `USAGE` significa “sin privilegios globales”.

Privilegio directo frente a privilegio heredado

En la actividad 2 los privilegios dejaron de otorgarse a cada usuario y pasaron a otorgarse a los roles. La Tabla 8 compara ambos mecanismos.

Tabla 8

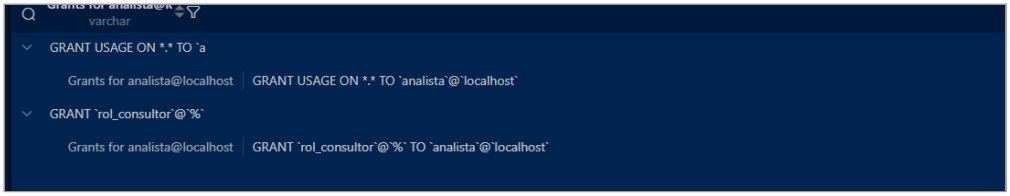
Comparación entre privilegio directo y privilegio heredado por rol

Aspecto	**Directo (`auditor`)**	**Heredado por rol (`analista`, `operador`, `admin_bd`)**
Cómo se otorga	`GRANT SELECT ON Guia_3.auditoria_log TO 'auditor'@'localhost'`	`GRANT ... TO 'rol_consultor'` y luego `GRANT 'rol_consultor' TO 'analista'@'localhost'`
Qué muestra `SHOW GRANTS`	El privilegio sobre la tabla	Solo la membresía: `GRANT` `` `rol_consultor`@`%` `` `TO` `` `analista`@`localhost` ``
Cómo ver los permisos reales	`SHOW GRANTS FOR usuario`	`SHOW GRANTS FOR usuario USING 'rol_consultor'`
Efecto de un `REVOKE`	Afecta solo a ese usuario	`REVOKE ... FROM 'rol_operador'` afecta a todos los miembros del rol al mismo tiempo
Mantenimiento	Una instrucción por usuario y por tabla	Se cambia el rol una vez y el cambio se propaga
Cuándo conviene	Excepciones puntuales, como una función única e independiente	Perfiles repetibles: varios analistas u operadores

Para que la demostración fuera clara, en la actividad 2 se **retiraron los privilegios directos** que la actividad 1 había dado a `admin_bd`, `analista` y `operador`. A partir de ese momento sus permisos vienen solo del rol. `auditor` conservó su privilegio directo a propósito, como punto de contraste. La propagación también se comprobó: un `REVOKE INSERT ON Guia_3.Programa FROM 'rol_operador'` le quitó de inmediato ese permiso a `operador`, sin tocar su cuenta. Las Figuras 5 a 7 muestran el resultado.

Figura 5

Privilegios de analista después de la actividad 2: solo la membresía al rol



Nota. `SHOW GRANTS` ya no lista los `SELECT` por tabla, sino `GRANT` `` `rol_consultor`@`%` ``. Los permisos efectivos se consultan con `USING 'rol_consultor'`.

Figura 6

Privilegios de operador después de la actividad 2

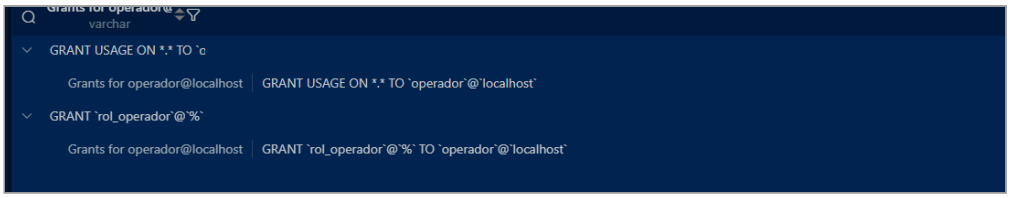
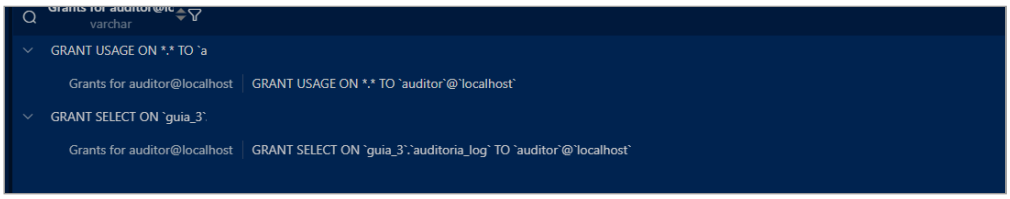


Figura 7

Privilegio de auditor después de la actividad 2: sigue siendo directo



Nota. Los cambios en los roles no afectan a `auditor`, porque su permiso no depende de ningún rol.

Mecanismos de auditoría

La auditoría del sistema se apoya en la tabla `auditoria_log` (Tabla 9) y en dos triggers sobre `Estudiante` (Tabla 10).

Tabla 9

Estructura de la tabla auditoria_log

Columna	Tipo	Qué registra
`id`	`INT AUTO_INCREMENT` (PK)	Consecutivo del evento
`tabla`	`VARCHAR(64)`	Tabla afectada (`Estudiante`)
`operacion`	`ENUM('UPDATE','DELETE')`	Tipo de cambio
`usuario`	`VARCHAR(100)`	Cuenta que ejecutó el cambio (`USER()`, por ejemplo `operador@localhost`)
`fecha`	`DATETIME`	Momento del cambio (`NOW()`)
`dato_anterior`	`JSON`	Fila completa antes del cambio (valores `OLD.*`)

Tabla 10

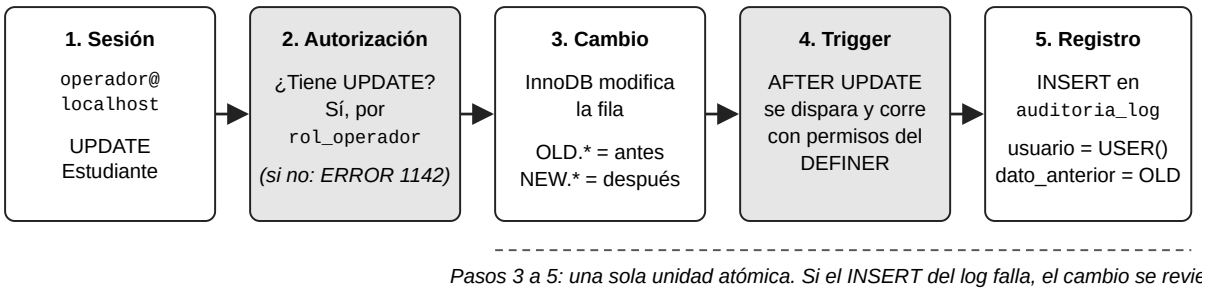
Triggers de auditoría instalados

Trigger	Momento	Evento	Tabla vigilada	Se ejecuta como
`trg_auditoria_estudiante_update`	`AFTER`	`UPDATE`	`Estudiante`	`root@localhost` (DEFINER)
`trg_auditoria_estudiante_delete`	`AFTER`	`DELETE`	`Estudiante`	`root@localhost` (DEFINER)

La Figura 8 describe paso a paso lo que ocurre cuando el operador modifica un estudiante.

Figura 8

Flujo de un evento auditado



Por qué `AFTER` y no `BEFORE`. El trigger `AFTER` solo se ejecuta si la operación sobre la fila tuvo éxito. Si el `UPDATE` falla por una restricción o por falta de privilegios, no queda un evento falso en el log. Además, en tablas transaccionales (InnoDB) el trigger y la sentencia forman una sola unidad: si el `INSERT` en el log falla, el cambio se revierte (Oracle, 2024e). Así no puede haber cambios sin rastro.

`USER()` frente a `CURRENT_USER()` (corrección aplicada). Dentro de un trigger, `CURRENT_USER()` devuelve la cuenta `DEFINER`, es decir, quien creó el trigger (`root`), y no quien hizo el cambio (Oracle, 2024c). Con esa función todos los eventos quedaban registrados como `root@localhost` y la columna `usuario` no servía para identificar responsables. La Figura 10

muestra ese problema en la primera ejecución. El script corregido (Anexo C) usa `USER()`, que devuelve la cuenta de la sesión que ejecutó la sentencia. También se completó el JSON con las columnas `PrefijoTelefono` y `FechaNacimiento`, que antes faltaban.

Figura 9

Tabla auditoria_log recién creada y vacía

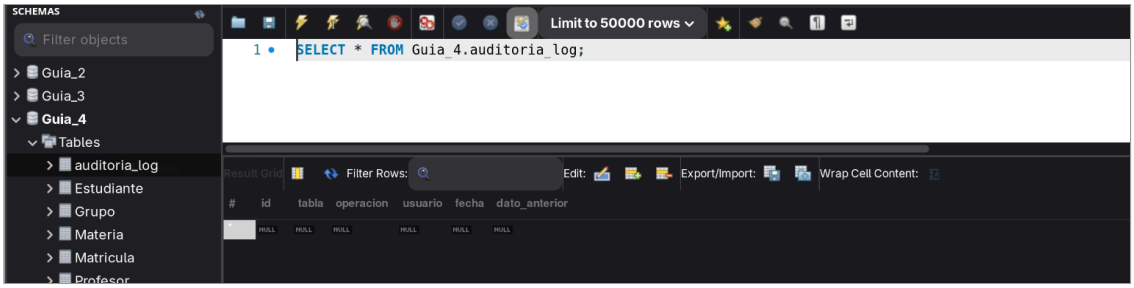
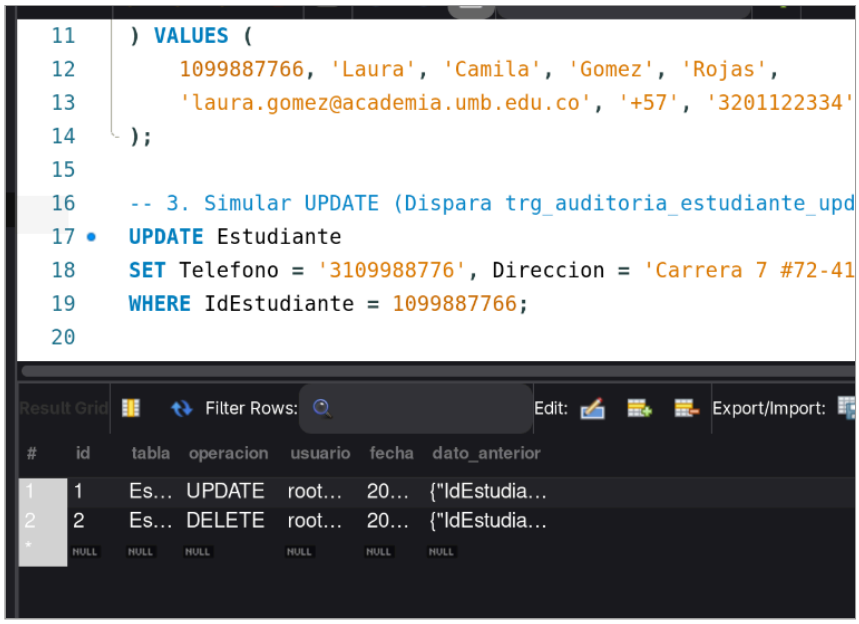


Figura 10

Primera ejecución de los triggers: eventos registrados a nombre de root



Nota. Los dos eventos (`UPDATE` y `DELETE`) quedaron con `usuario = root`… porque la versión inicial usaba `CURRENT_USER()`. Esta evidencia dio origen al hallazgo H3.

Con la corrección, cada evento identifica a su responsable. El siguiente es el valor de `dato_anterior` registrado cuando `operador@localhost` cambió el teléfono y la dirección del estudiante de prueba:

```
{
  "IdEstudiante": 1099887766, "PrimerNombre": "Laura", "SegundoNombre": "Camila",
  "PrimerApellido": "Gomez", "SegundoApellido": "Rojas",
  "CorreoEstudiante": "laura.gomez@academia.umb.edu.co",
  "PrefijoTelefono": "+57", "Telefono": "3201122334",
  "FechaNacimiento": "2004-06-20", "Direccion": "Calle 100 #15-20"
}
```

Con este JSON es posible **reconstruir** el valor previo de cualquier fila modificada o eliminada, no solo saber que cambió. Por ejemplo, `dato_anterior->>'$.Telefono'` devuelve el teléfono que había antes del cambio.

Control en la capa de aplicación

La actividad 3 escribió dos consultas en Python vulnerables a inyección y luego las reescribió con consultas parametrizadas. La Tabla 11 compara ambas versiones.

Tabla 11

Consulta vulnerable frente a consulta parametrizada

	Antes (vulnerable)	**Después (seguro)**
Construcción	`f"... WHERE IdEstudiante = '{entrada}'"`	`"... WHERE IdEstudiante = %s"` + tupla de parámetros
Caso 1: entrada `1031422990' OR '1'='1`	El `OR '1'='1'` se ejecuta como SQL y devuelve **toda** la tabla (5 de 5 estudiantes)	Se trata como un valor literal: devuelve como máximo el registro exacto (1 fila)
Caso 2: entrada `1031422990 OR 1=1 --`	El `--` comenta el `AND IdMatricula = ...`, se pierde el filtro y se exponen las 13 matrículas	El `AND` se mantiene; la entrada no altera la estructura (0 filas)
Por qué funciona	Datos y código viajan mezclados en un solo texto	La estructura de la consulta queda fija antes de recibir los datos; los datos nunca se interpretan como código

Nota. Las cantidades de filas se verificaron al volver a ejecutar ambas versiones con `mysql-connector-python` sobre los datos de la práctica.

Validación de entrada como segunda línea de defensa. Al comparar una cadena con una columna `INT`, MySQL la convierte a número tomando los dígitos iniciales (`'1031422990'' OR ...'` se convierte en `1031422990`). Por eso, en el caso 1, la versión segura devuelve el registro de ese estudiante y no cero filas: no hay inyección, pero sí una conversión implícita que puede sorprender. La consulta parametrizada ya impide la inyección. Además, validar el tipo antes de consultar (`entrada.isdigit()` y longitud de 6 a 10 dígitos) rechaza las entradas malformadas antes de que lleguen a la base de datos (Anexo D). OWASP Foundation (2023) recomienda combinar ambas medidas.

Evidencias del mapa de seguridad

El script `05_Actividad5_Mapa_Seguridad.sql` (Anexo A) reúne en nueve bloques las consultas que demuestran el mapa. Se ejecuta como `root` y cada bloque genera una evidencia. La Tabla 12 indica qué demuestra cada uno. El Anexo B contiene la salida de referencia.

Tabla 12

Bloques del script 05 y lo que demuestran

Bloque	Fuente consultada	Qué demuestra
1	`mysql.user`	Inventario de cuentas: 4 usuarios desbloqueados y 3 roles bloqueados
2	`mysql.role_edges`, `mysql.default_roles`	Asignación rol → usuario y roles activos por defecto
3	`information_schema.SCHEMA_PRIVILEGES`	Privilegios de esquema: solo `rol_admin` tiene `Guia_3.*`
4	`mysql.tables_priv`	Privilegios por tabla (matriz de la Tabla 6)
5	`SHOW GRANTS ... [USING]`	Diferencia entre privilegio directo y heredado
6	`mysql.user`, `mysql.db`, `mysql.tables_priv`	Cuentas con `GRANT OPTION` (insumo del hallazgo H2)
7	`information_schema.TRIGGERS`	Triggers de auditoría instalados y su `DEFINER`
8	`Guia_3.auditoria_log`	Resumen y detalle del log: quién, qué y cuándo
9	Variables del servidor, `mysql.user`	Postura del servidor: cuentas anónimas, host `%`, política de claves, TLS

Nota. El script solo **consulta** el catálogo. No modifica la configuración del SGBD, en línea con la recomendación 5 del laboratorio.

Validación técnica del mapa

Para comprobar que el informe describe correctamente el sistema, se reprodujo el entorno completo en un servidor MySQL 8.0.46 de prueba. Se ejecutaron en orden los scripts de estructura, datos, actividad 1, actividad 2, los triggers corregidos y el script 05. Luego se repitieron las pruebas de acceso con cada cuenta. La Tabla 13 resume los resultados.

Tabla 13

Pruebas de validación ejecutadas y resultado obtenido

#	Cuenta	Prueba	Resultado
1	`analista`	`SELECT COUNT(*) FROM Estudiante`	OK (5 filas)
2	`analista`	`INSERT INTO Programa ...`	`ERROR 1142`: *INSERT command denied*
3	`analista`	`SELECT ... FROM auditoria_log`	`ERROR 1142`
4	`operador`	`UPDATE Estudiante SET Telefono ...`	OK; el trigger registra `operador@localhost`
5	`operador`	`DELETE FROM Estudiante ...`	`ERROR 1142`; no se genera evento en el log
6	`operador`	`SELECT ... FROM auditoria_log`	`ERROR 1142`
7	`admin_bd`	`DELETE FROM Estudiante ...`	OK; el trigger registra `admin_bd@localhost`
8	`auditor`	`SELECT ... FROM auditoria_log`	OK (2 eventos con responsable correcto)
9	`auditor`	`SELECT ... FROM Estudiante` / `INSERT INTO auditoria_log`	`ERROR 1142` en ambos casos
10	`admin_bd`	`DELETE FROM auditoria_log WHERE id = 1`	**OK: el DBA puede borrar el log (confirma H2)**
11	—	Recomendaciones R2, R4, R5, R6, R7, R8 y R11 en el servidor de prueba	Sintaxis válida; ver detalle abajo

La prueba 5 confirma el comportamiento de los triggers `AFTER`: el borrado rechazado no dejó ningún evento en el log. La prueba 10 confirma el riesgo más importante del sistema: con los permisos actuales, `admin_bd` puede borrar la evidencia de sus propias acciones. Para las recomendaciones de producción se verificó en el servidor de prueba lo siguiente: (a) un trigger cuyo `DEFINER` es una cuenta bloqueada se ejecuta normalmente y escribe en un esquema `auditoria` que `admin_bd` ya no puede leer; (b) los triggers de inmutabilidad rechazan `UPDATE` y `DELETE` sobre el

log con `ERROR 1644`, aunque **no** impiden un `TRUNCATE`, que no dispara triggers; (c) la vista enmascarada muestra `******1771` en lugar del teléfono completo, mientras que la tabla base responde `ERROR 1142` al analista; y (d) `REVOKE GRANT OPTION` sobre `rol_admin` elimina la capacidad de delegar. Estas observaciones ajustaron las recomendaciones R6 y R11.

Análisis de riesgos

Los hallazgos se clasificaron según su severidad. **Alta**: permite fraude, fuga masiva o pérdida de evidencia. **Media**: facilita un ataque o expone datos personales. **Baja**: buena práctica pendiente. La Tabla 14 presenta los once hallazgos.

Tabla 14

Hallazgos del análisis de riesgos

ID	Hallazgo	Riesgo concreto	Severidad	Estado
H1	Contraseñas en texto plano en `01_Actividad1_Usuarios_Privilegios.sql` y en el README	Cualquier persona con acceso al repositorio o a la entrega obtiene las credenciales. Incumple la recomendación 11 de la guía	**Alta**	Abierto; aceptable solo en laboratorio
H2	`admin_bd` tiene `ALL` + `GRANT OPTION` sobre `Guia_3.*`, incluida `auditoria_log`	El administrador puede borrar o alterar el log que registra sus propios cambios, o ejecutar `DROP TRIGGER`. No hay separación de funciones (validado, prueba 10)	**Alta**	Abierto
H3	El trigger registraba `CURRENT_USER()` (el DEFINER)	Todos los eventos aparecían como `root`: la auditoría no identificaba responsables	**Alta**	**Corregido** (`USER()`)
H4	Los scripts de la actividad 3 se conectan como `root` por defecto	Si la aplicación tuviera una inyección no detectada, el atacante tendría control total del servidor	**Alta**	Abierto

ID	Hallazgo	Riesgo concreto	Severidad	Estado
H5	La auditoría cubre solo `Estudiante`, solo `UPDATE`/`DELETE`, y no guarda el valor nuevo	Los cambios en `Matricula` (notas, valores) y las inserciones fraudulentas no dejan rastro. `TRUNCATE` no dispara triggers `DELETE`	Media	Abierto
H6	`analista` ve todos los datos personales: teléfono, dirección, correo, fecha de nacimiento, notas y valores	Exposición de datos personales sin necesidad de negocio (Ley 1581 de 2012, principio de acceso y circulación restringida)	Media	Abierto
H7	Sin política de contraseñas, vencimiento ni bloqueo por intentos fallidos	Las contraseñas débiles o reutilizadas son vulnerables a fuerza bruta	Media	Abierto
H8	No se exige conexión cifrada (TLS)	En red, las credenciales y los datos viajan en claro. Mitigado parcialmente por `@'localhost'`	Media	Abierto
H9	`auditoria_log` vive en el mismo esquema que los datos y no tiene política de retención	Un `ALL ON Guia_3.*` siempre lo incluirá; el log crece sin control	Baja	Abierto
H10	`fecha` usa `NOW()` (hora local del servidor)	Ambigüedad al correlacionar eventos entre servidores o tras un cambio de zona horaria	Baja	Abierto
H11	Las entradas no se validaban antes de consultar	Comportamiento inesperado por conversión implícita de tipos	Baja	**Corregido** (validación `isdigit()`)

La Tabla 15 ubica los hallazgos abiertos según su probabilidad y su impacto, como apoyo para priorizar.

Tabla 15

Matriz de probabilidad e impacto de los hallazgos abiertos

Probabilidad \ Impacto	Bajo	Medio	Alto
Alta		H7	**H1, H2**
Media	H10	H6, H8	**H4, H5**
Baja	H9		

Nota. Valoración cualitativa del equipo para un despliegue en red institucional. Los hallazgos corregidos (H3 y H11) no se incluyen.

Recomendaciones para producción

Las recomendaciones siguen el orden de las capas de la Tabla 3, y cada una indica qué hallazgo resuelve. Los fragmentos SQL son para MySQL 8.0.19 o superior. No se ejecutaron en el entorno del laboratorio porque cambian la configuración del SGBD, pero su sintaxis se validó en el servidor de prueba.

Autenticación

R1. Ninguna contraseña en el código (H1). La contraseña se genera en el servidor y se entrega por un canal seguro. En la aplicación se lee de un gestor de secretos o de variables de entorno:

```
-- MySQL genera la clave y la muestra UNA sola vez; nunca queda escrita en el script
CREATE USER 'operador'@'localhost' IDENTIFIED BY RANDOM PASSWORD;
```

```
# Aplicacion: la clave sale del entorno, no del codigo fuente
import os
password = os.environ["DB_PASSWORD"]
```

R2. Política de contraseñas, vencimiento y bloqueo (H7). Se instala el componente de validación y se fijan reglas por cuenta (Oracle, 2024d):

```
INSTALL COMPONENT 'file://component_validate_password';
SET PERSIST validate_password.policy = 'STRONG';   -- mayusculas, numeros, simbolos
SET PERSIST validate_password.length = 12;

ALTER USER 'operador'@'localhost'
    PASSWORD EXPIRE INTERVAL 90 DAY      -- cambio obligatorio cada 90 dias
    PASSWORD HISTORY 5                   -- no reutilizar las 5 anteriores
    FAILED_LOGIN_ATTEMPTS 3              -- 3 intentos fallidos...
    PASSWORD_LOCK_TIME 1;                -- ...bloquean la cuenta 1 dia
```

R3. Proteger `root` y no usarlo (H4). Se ejecuta `mysql_secure_installation`, que elimina las cuentas anónimas y la base `test` y limita `root` a `localhost`. `root` se reserva para emergencias; ninguna aplicación ni persona lo usa en el día a día.

R4. Cifrado en tránsito (H8).

```
SET PERSIST require_secure_transport = ON;          -- rechaza conexiones TCP sin TLS
ALTER USER 'admin_bd'@'localhost' REQUIRE SSL;
```

Autorización

R5. Cuenta propia para la aplicación (H4). La aplicación se conecta con una cuenta de mínimo privilegio, nunca con `root`. Esa cuenta se limita al host o subred del servidor de aplicaciones y a un número máximo de conexiones:

```
CREATE USER 'app_academico'@'10.0.0.%' IDENTIFIED BY RANDOM PASSWORD
    REQUIRE SSL WITH MAX_USER_CONNECTIONS 20;
GRANT 'rol_operador' TO 'app_academico'@'10.0.0.%';
SET DEFAULT ROLE 'rol_operador' TO 'app_academico'@'10.0.0.%';
```

R6. Separación de funciones: sacar el log del alcance del DBA (H2, H9). `auditoria_log` se mueve a un esquema propio (`auditoria`) sobre el que `rol_admin` no tiene permisos. Los triggers usan como `DEFINER` una cuenta técnica bloqueada que solo puede insertar en el log:

```
CREATE DATABASE auditoria;
CREATE TABLE auditoria.auditoria_log LIKE Guia_3.auditoria_log;
ALTER TABLE auditoria.auditoria_log
    ADD COLUMN dato_nuevo JSON NULL,
    ADD COLUMN host VARCHAR(255) NULL;

CREATE USER 'trg_auditoria'@'localhost' ACCOUNT LOCK;     -- nadie puede iniciar sesion
con ella
GRANT INSERT ON auditoria.auditoria_log TO 'trg_auditoria'@'localhost';
GRANT SELECT, TRIGGER ON Guia_3.Estudiante TO 'trg_auditoria'@'localhost';

-- Los triggers se recrean asi (ejecuta root o una cuenta con SET_USER_ID):
-- CREATE DEFINER = 'trg_auditoria'@'localhost' TRIGGER trg_auditoria_estudiante_update
--   AFTER UPDATE ON Guia_3.Estudiante FOR EACH ROW
--   INSERT INTO auditoria.auditoria_log (...) VALUES (..., USER(), UTC_TIMESTAMP(), ...);

GRANT SELECT ON auditoria.auditoria_log TO 'auditor'@'localhost';
```

Con esto `admin_bd` sigue administrando `Guia_3.*`, pero ya no puede tocar `auditoria.*`. Solo el auditor lee el log y solo los triggers escriben en él. Queda un riesgo residual que debe documentarse: `admin_bd` todavía tiene el privilegio `TRIGGER` sobre `Guia_3` y podría eliminar los triggers. Ese evento lo captura la auditoría del servidor (R12), o puede evitarse retirando `TRIGGER` de `rol_admin` y dejando los cambios de triggers a un proceso con doble aprobación.

R7. Retirar `GRANT OPTION` del rol administrativo (H2). La delegación de permisos queda en una única cuenta DBA nominal, para que cada cambio de permisos tenga un responsable identificable (Joint Task Force, 2020, control AC-2):

```
REVOKE GRANT OPTION ON Guia_3.* FROM 'rol_admin';
```

R8. Vistas y permisos por columna para datos personales (H6). El analista trabaja con una vista que oculta o enmascara los datos sensibles en lugar de la tabla base. Las vistas son un mecanismo de seguridad clásico (Elmasri & Navathe, 2016, cap. 30):

```
CREATE SQL SECURITY DEFINER VIEW Guia_3.vw_estudiante_consulta AS
SELECT IdEstudiante,
       PrimerNombre, PrimerApellido,
       CONCAT('******', RIGHT(Telefono, 4))          AS Telefono,
       CONCAT(LEFT(CorreoEstudiante, 2), '***@***')  AS Correo
FROM   Guia_3.Estudiante;

REVOKE SELECT ON Guia_3.Estudiante FROM 'rol_consultor';
GRANT  SELECT ON Guia_3.vw_estudiante_consulta TO 'rol_consultor';

-- Alternativa: privilegio por columna
GRANT SELECT (IdEstudiante, PrimerNombre, PrimerApellido) ON Guia_3.Estudiante TO
'rol_consultor';
```

R9. Revisión periódica de accesos. El script `05_Actividad5_Mapa_Seguridad.sql` se ejecuta cada trimestre y cada vez que alguien cambia de cargo o sale de la organización. Las cuentas que ya no se usan se bloquean (`ALTER USER ... ACCOUNT LOCK`) antes de eliminarse, para conservar la trazabilidad mientras se confirma que nadie las necesita.

Auditoría

R10. Ampliar la cobertura (H5). Se crean triggers `AFTER INSERT`, `AFTER UPDATE` y `AFTER DELETE` en las tablas críticas, en especial `Matricula`, porque contiene `NotaFinal` y `ValorMatricula`, objetivos típicos de fraude. Se agregan las columnas `dato_nuevo JSON` (valores `NEW.*`) y `host` para ver el antes y el después de cada cambio. La columna `operacion` se amplía a `ENUM('INSERT','UPDATE','DELETE')`.

R11. Hacer el log inalterable. Aun con el permiso adecuado, nadie debería poder modificar ni borrar un evento. El nombre del trigger se califica con el esquema, porque un trigger debe crearse en el mismo esquema que su tabla:

```
-- Cuerpo de una sola sentencia: no requiere DELIMITER
CREATE TRIGGER auditoria.trg_log_inmutable_upd BEFORE UPDATE ON auditoria.auditoria_log
FOR EACH ROW SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'auditoria_log es de solo
insercion';

CREATE TRIGGER auditoria.trg_log_inmutable_del BEFORE DELETE ON auditoria.auditoria_log
FOR EACH ROW SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'auditoria_log es de solo
insercion';
```

La validación mostró que `TRUNCATE TABLE` vacía el log sin disparar estos triggers. `TRUNCATE` requiere el privilegio `DROP`, así que ninguna cuenta, salvo la de emergencia, debe tener `DROP` sobre el esquema `auditoria`. Esto se cumple con R6, porque `trg_auditoria` solo tiene

`INSERT` y `auditor` solo `SELECT`.

R12. Auditar lo que los triggers no ven. Los triggers no capturan `SELECT`, `TRUNCATE`, `DROP`, cambios de permisos ni inicios de sesión fallidos. En producción conviene un plugin de auditoría del servidor (MySQL Enterprise Audit, Percona Audit Log Plugin o `server_audit` en MariaDB) que envíe los eventos a un sistema externo (SIEM) fuera del alcance del DBA (Joint Task Force, 2020, controles AU-2 y AU-9).

R13. Hora en UTC y retención (H10, H9). Los triggers usan `UTC_TIMESTAMP()`. Se define cuánto tiempo se conserva el log (por ejemplo, 5 años) y se archiva por particiones mensuales (Joint Task Force, 2020, controles AU-8 y AU-11).

Aplicación

R14. Consultas parametrizadas obligatorias. Ninguna consulta se arma concatenando texto (actividad 3). Esto se verifica en la revisión de código y, de forma automática, con herramientas de análisis estático como Bandit, que detecta SQL construido con *f-strings*.

R15. Validar entradas con lista blanca (H11). Se validan tipo, longitud y formato antes de consultar. Por ejemplo, el documento debe ser numérico y tener de 6 a 10 dígitos (Joint Task Force, 2020, control SI-10).

R16. Mensajes de error genéricos. Al usuario se le muestra “Error al procesar la solicitud”. El error SQL detallado va solo al log interno, porque revela nombres de tablas y columnas que le sirven a un atacante (Joint Task Force, 2020, control SI-11).

Operación e infraestructura

R17. Red. Se configura `bind-address` solo con la interfaz necesaria y se abre el puerto 3306 en el firewall únicamente para el servidor de aplicaciones. Se desactiva `local_infile` (`SET PERSIST local_infile = OFF`) para impedir la lectura de archivos del cliente.

R18. Copias de seguridad. Se programa un `mysqldump --single-transaction --routines --triggers` diario, se cifra la copia, se guarda fuera del servidor y se **prueba la restauración** cada mes. Se activan los *binary logs* para poder recuperar el estado de un momento puntual.

R19. Actualizaciones. Se aplican cada trimestre los parches de seguridad de MySQL que Oracle publica en sus *Critical Patch Updates*.

Hoja de ruta priorizada

La Tabla 16 ordena las recomendaciones en tres horizontes de tiempo.

Tabla 16

Hoja de ruta de implementación

Prioridad	Plazo	Recomendaciones	Hallazgos que cierra
1	Antes de salir a producción	R1, R3, R5, R6, R7, R14	H1, H2, H4
2	Primer mes	R2, R4, R8, R10, R11, R15, R16	H5, H6, H7, H8, H11
3	Primer trimestre	R9, R12, R13, R17, R18, R19	H9, H10

Correspondencia con controles NIST SP 800-53

Para facilitar una futura revisión formal, la Tabla 17 relaciona cada recomendación con el control equivalente del catálogo NIST SP 800-53 Rev. 5 (Joint Task Force, 2020).

Tabla 17

Recomendaciones y controles NIST SP 800-53 Rev. 5 relacionados

Recomendación	Control	Nombre del control
R1, R2	IA-5, AC-7	Gestión de autenticadores; intentos de inicio de sesión fallidos
R3, R5, R7, R9	AC-2, AC-6	Gestión de cuentas; mínimo privilegio
R4	SC-8	Confidencialidad e integridad de la transmisión
R6	AC-5, AU-9	Separación de funciones; protección de la información de auditoría
R8	AC-3	Aplicación del control de acceso
R10, R12	AU-2, AU-3, AU-12	Registro de eventos; contenido de los registros; generación de registros
R11	AU-9	Protección de la información de auditoría
R13	AU-8, AU-11	Marcas de tiempo; retención de registros de auditoría
R14, R15	SI-10	Validación de la información de entrada
R16	SI-11	Manejo de errores
R17	SC-7, CM-7	Protección de límites; funcionalidad mínima
R18	CP-9	Copia de seguridad del sistema
R19	SI-2	Corrección de fallas

Respuestas a las preguntas orientadoras

¿Cuál es la diferencia entre autenticación y autorización?

La autenticación verifica **quién** es el usuario mediante la cuenta, el host y la contraseña (`CREATE USER`). La autorización decide **qué** puede hacer ese usuario ya identificado (`GRANT`/`REVOKE` y roles). Son etapas sucesivas: sin autenticación no hay sesión, y con sesión cada sentencia pasa por la autorización. En este sistema, `analista` se autentica correctamente, pero la autorización le niega el `INSERT` con `ERROR 1142`.

¿Qué riesgos tiene usar `root` en producción?

`root` tiene privilegios globales: puede leer, borrar o alterar cualquier base, crear cuentas, desactivar la auditoría y leer archivos del servidor. Si la aplicación usa `root` y sufre una inyección SQL, el atacante obtiene todo eso. Además, todos los cambios quedan a nombre de la misma cuenta, así que se pierde la trazabilidad (hallazgos H3 y H4). Por último, un error humano, como un `DROP` o un `UPDATE` sin `WHERE`, no encuentra ningún límite.

¿Cómo funciona la inyección SQL y qué la previene?

Ocurre cuando la entrada del usuario se concatena dentro del texto de la consulta y el motor la interpreta como código (`' OR '1'='1`, `--`). En la actividad 3, esa técnica expuso toda la tabla `Estudiante` y las 13 matrículas. Se previene con consultas parametrizadas, que separan la estructura de los datos. Se complementa con validación de entradas y con una cuenta de mínimo privilegio que limite el daño si algo falla (OWASP Foundation, 2023).

¿Qué es un trigger de auditoría y cómo rastrea cambios?

Es un procedimiento que el motor ejecuta automáticamente ante un evento (`UPDATE`, `DELETE`) sobre una tabla. Guarda en `auditoria_log` quién hizo el cambio (`USER()`), cuándo (`NOW()`) y cuál era el valor anterior (`OLD.*` en formato JSON). Como lo ejecuta el motor y no la aplicación, no se puede omitir desde el cliente, y como corre con los permisos de su `DEFINER`, el usuario que dispara el evento no necesita ni obtiene permiso para escribir en el log.

Conclusiones

La combinación de cuentas atadas a `localhost`, permisos por tabla y roles permitió aplicar el principio de mínimo privilegio. Cada perfil puede hacer exactamente su trabajo, y las pruebas confirmaron con `ERROR 1142` todo lo que queda prohibido, tanto en el laboratorio como en la validación posterior.

Los roles simplifican la administración: un solo `REVOKE` sobre `rol_operador` cambia los permisos de todos los operadores a la vez. El privilegio directo queda para casos excepcionales, como el del `auditor`, cuya independencia conviene que no dependa de cambios en los roles.

Las consultas parametrizadas eliminan la inyección SQL de raíz, porque los datos nunca se interpretan como código. La validación de entradas y una cuenta de aplicación sin privilegios administrativos completan la defensa.

La auditoría por triggers responde quién, qué, cuándo y qué había antes. La práctica también reveló un detalle técnico que la invalidaba: `CURRENT_USER()` registra al `DEFINER` del trigger y no a quien hizo el cambio. Se corrigió con `USER()`, y la validación confirmó que ahora cada evento identifica a su responsable.

El riesgo más importante que queda es la falta de separación de funciones: el DBA puede alterar el log que lo vigila, y la validación lo comprobó. En producción la auditoría debe vivir fuera de su alcance, ser de solo inserción y complementarse con una auditoría del servidor enviada a un sistema externo (R6, R11 y R12).

Referencias

Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Joint Task Force. (2020). *Security and privacy controls for information systems and organizations* (NIST Special Publication 800-53, Rev. 5). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-53r5

Ministerio de Comercio, Industria y Turismo. (2013). *Decreto 1377 de 2013, por el cual se reglamenta parcialmente la Ley 1581 de 2012*. Diario Oficial No. 48.834.

Oracle. (2024a). *MySQL 8.0 reference manual: Using roles*. https://dev.mysql.com/doc/refman/8.0/en/roles.html

Oracle. (2024b). *MySQL 8.0 security guide*. https://dev.mysql.com/doc/mysql-security-excerpt/8.0/en/

Oracle. (2024c). *MySQL 8.0 reference manual: Stored object access control*. https://dev.mysql.com/doc/refman/8.0/en/stored-objects-security.html

Oracle. (2024d). *MySQL 8.0 reference manual: Password management*. https://dev.mysql.com/doc/refman/8.0/en/password-management.html

Oracle. (2024e). *MySQL 8.0 reference manual: Using triggers*. https://dev.mysql.com/doc/refman/8.0/en/triggers.html

OWASP Foundation. (2021). *OWASP Top 10:2021 – A03:2021 Injection*. https://owasp.org/Top10/A03_2021-Injection/

OWASP Foundation. (2023). *SQL injection prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill.

Anexo A

Script `05_Actividad5_Mapa_Seguridad.sql`

El script se ejecuta como `root` y solo consulta el catálogo. Cada bloque corresponde a una fila de la Tabla 12.

```
-- ============================================================================
-- Practica 4 - Actividad 5: Mapa de seguridad del sistema Guia_3
-- SGBD: MySQL 8.0 | Ejecutar como root
-- El script SOLO CONSULTA el catalogo (mysql.*, information_schema).
-- No crea, modifica ni elimina cuentas, privilegios ni configuracion.
-- Cada bloque corresponde a una figura del informe (Figuras 5.1 a 5.9).
-- ============================================================================

-- ----------------------------------------------------------------------------
-- BLOQUE 1 (Fig. 5.1) - Inventario de cuentas: usuarios y roles
--   Un rol de MySQL 8 es una cuenta bloqueada y sin contrasena.
-- ----------------------------------------------------------------------------
SELECT  u.User                                  AS cuenta,
        u.Host                                  AS host,
        CASE WHEN u.account_locked = 'Y'
              AND u.authentication_string = ''  THEN 'ROL'
             ELSE 'USUARIO' END                 AS tipo,
        u.account_locked                        AS bloqueada,
        u.plugin                                AS plugin_autenticacion,
        u.password_last_changed                 AS clave_cambiada,
        IFNULL(u.password_lifetime, 'global')   AS vigencia_clave_dias
FROM    mysql.user u
WHERE   u.User IN ('admin_bd','analista','operador','auditor',
                   'rol_admin','rol_consultor','rol_operador')
ORDER BY tipo DESC, cuenta;

-- ----------------------------------------------------------------------------
-- BLOQUE 2 (Fig. 5.2) - Asignacion rol -> usuario y roles activos por defecto
-- ----------------------------------------------------------------------------
SELECT  CONCAT(e.TO_USER, '@', e.TO_HOST)       AS usuario,
        CONCAT(e.FROM_USER, '@', e.FROM_HOST)   AS rol_asignado,
        e.WITH_ADMIN_OPTION                     AS puede_reasignar_rol,
        IF(d.USER IS NULL, 'NO', 'SI')          AS activo_por_defecto
FROM    mysql.role_edges e
LEFT JOIN mysql.default_roles d
       ON d.USER = e.TO_USER AND d.HOST = e.TO_HOST
      AND d.DEFAULT_ROLE_USER = e.FROM_USER AND d.DEFAULT_ROLE_HOST = e.FROM_HOST
ORDER BY usuario;

-- ----------------------------------------------------------------------------
-- BLOQUE 3 (Fig. 5.3) - Privilegios a nivel de esquema (Guia_3.*)
--   Esperado: solo rol_admin aparece aqui.
-- ----------------------------------------------------------------------------
SELECT  GRANTEE                                 AS cuenta,
        TABLE_SCHEMA                            AS esquema,
        GROUP_CONCAT(PRIVILEGE_TYPE ORDER BY PRIVILEGE_TYPE SEPARATOR ', ')
                                                AS privilegios,
        MAX(IS_GRANTABLE)                       AS grant_option
FROM    information_schema.SCHEMA_PRIVILEGES
WHERE   TABLE_SCHEMA = 'Guia_3'
```

```
GROUP BY GRANTEE, TABLE_SCHEMA;

-- ------------------------------------------------------------------------
-- BLOQUE 4 (Fig. 5.4) - Privilegios a nivel de tabla (matriz de la seccion 3.4)
-- ------------------------------------------------------------------------
SELECT  CONCAT(User, '@', Host)                AS cuenta,
        Table_name                             AS tabla,
        Table_priv                             AS privilegios,
        Grantor                                AS otorgado_por
FROM    mysql.tables_priv
WHERE   Db = 'Guia_3'
ORDER BY cuenta, tabla;

-- Vista pivote: una fila por tabla, una columna por cuenta
SELECT  t.TABLE_NAME AS tabla,
        IFNULL(MAX(CASE WHEN tp.User='rol_consultor' THEN tp.Table_priv END), '-') AS
rol_consultor,
        IFNULL(MAX(CASE WHEN tp.User='rol_operador'  THEN tp.Table_priv END), '-') AS
rol_operador,
        IFNULL(MAX(CASE WHEN tp.User='auditor'       THEN tp.Table_priv END), '-') AS
auditor
FROM    information_schema.TABLES t
LEFT JOIN mysql.tables_priv tp
       ON tp.Db = t.TABLE_SCHEMA AND tp.Table_name = t.TABLE_NAME
WHERE   t.TABLE_SCHEMA = 'Guia_3'
GROUP BY t.TABLE_NAME
ORDER BY t.TABLE_NAME;

-- ------------------------------------------------------------------------
-- BLOQUE 5 (Fig. 5.5) - Privilegio directo vs. heredado
-- ------------------------------------------------------------------------
SHOW GRANTS FOR 'auditor'@'localhost';                              -- directo
SHOW GRANTS FOR 'analista'@'localhost';                             -- solo membresia
SHOW GRANTS FOR 'analista'@'localhost' USING 'rol_consultor';       -- efectivos
SHOW GRANTS FOR 'operador'@'localhost' USING 'rol_operador';
SHOW GRANTS FOR 'admin_bd'@'localhost' USING 'rol_admin';

-- ------------------------------------------------------------------------
-- BLOQUE 6 (Fig. 5.6) - Cuentas que pueden delegar privilegios (GRANT OPTION)
--   Insumo del hallazgo H2.
-- ------------------------------------------------------------------------
SELECT 'global'  AS nivel, CONCAT(User,'@',Host) AS cuenta, '*.*' AS objeto
FROM   mysql.user WHERE Grant_priv = 'Y'
UNION ALL
SELECT 'esquema', CONCAT(User,'@',Host), CONCAT(Db,'.*')
FROM   mysql.db   WHERE Grant_priv = 'Y' AND Db = 'Guia_3'
UNION ALL
SELECT 'tabla',   CONCAT(User,'@',Host), CONCAT(Db,'.',Table_name)
FROM   mysql.tables_priv WHERE FIND_IN_SET('Grant', Table_priv) AND Db = 'Guia_3';

-- ------------------------------------------------------------------------
-- BLOQUE 7 (Fig. 5.7) - Triggers de auditoria instalados y su DEFINER
-- ------------------------------------------------------------------------
SELECT  TRIGGER_NAME        AS trigger_nombre,
        ACTION_TIMING       AS momento,
        EVENT_MANIPULATION  AS evento,
        EVENT_OBJECT_TABLE  AS tabla_vigilada,
        DEFINER             AS se_ejecuta_como,
        CREATED             AS creado
```

```
FROM    information_schema.TRIGGERS
WHERE   TRIGGER_SCHEMA = 'Guia_3'
ORDER BY tabla_vigilada, evento;

-- ============================================================================
-- BLOQUE 8 (Fig. 5.8) - Rastro de auditoria: resumen y detalle
-- ============================================================================
SELECT  tabla, operacion, usuario,
        COUNT(*)   AS eventos,
        MIN(fecha) AS primer_evento,
        MAX(fecha) AS ultimo_evento
FROM    Guia_3.auditoria_log
GROUP BY tabla, operacion, usuario
ORDER BY tabla, operacion;

SELECT  id, tabla, operacion, usuario, fecha,
        dato_anterior->>'$.IdEstudiante' AS id_afectado,
        dato_anterior->>'$.Telefono'     AS telefono_anterior,
        dato_anterior->>'$.Direccion'    AS direccion_anterior
FROM    Guia_3.auditoria_log
ORDER BY id;

-- ============================================================================
-- BLOQUE 9 (Fig. 5.9) - Postura general del servidor
-- ============================================================================
-- 9.1 Cuentas anonimas (deben ser 0)
SELECT COUNT(*) AS cuentas_anonimas FROM mysql.user WHERE User = '';

-- 9.2 Cuentas de usuario que aceptan conexion desde cualquier host
--     (los roles aparecen con '%' pero estan bloqueados)
SELECT User, Host, account_locked
FROM   mysql.user
WHERE  Host = '%' AND account_locked = 'N';

-- 9.3 Cuentas desbloqueadas sin contrasena (deben ser 0, salvo las que usan
--     autenticacion por socket del sistema operativo, plugin auth_socket)
SELECT User, Host, plugin FROM mysql.user
WHERE  authentication_string = '' AND account_locked = 'N';

-- 9.4 Politica de contrasenas, cifrado y otras variables de seguridad
SHOW VARIABLES WHERE Variable_name IN (
    'default_password_lifetime', 'password_history', 'password_reuse_interval',
    'require_secure_transport', 'have_ssl', 'tls_version',
    'local_infile', 'bind_address', 'log_bin', 'activate_all_roles_on_login',
    'mandatory_roles', 'version'
) OR Variable_name LIKE 'validate_password%';

-- 9.5 Componentes instalados (validate_password, si existe)
SELECT component_urn FROM mysql.component;
-- ============================================================================
```

Anexo B

Salida de referencia del script 05

Salida obtenida al ejecutar el script en el servidor MySQL 8.0.46 de prueba, después de reproducir las actividades 1 a 4 y las pruebas de la Tabla 13. Las marcas de tiempo están en UTC. En el entorno del equipo los valores de fecha y versión pueden ser distintos, pero la estructura de cuentas, roles y privilegios debe coincidir.

Bloque 1. Inventario de cuentas

cuenta	host	tipo	bloqueada	plugin_autenticacion	clave_cambiada	vigencia_clave_dias
admin_bd	localhost	USUARIO	N	caching_sha2_password	2026-10-06 00:57:23	global
analista	localhost	USUARIO	N	caching_sha2_password	2026-10-06 00:57:23	global
auditor	localhost	USUARIO	N	caching_sha2_password	2026-10-06 00:57:23	global
operador	localhost	USUARIO	N	caching_sha2_password	2026-10-06 00:57:23	global
rol_admin	%	ROL	Y	caching_sha2_password	2026-10-06 00:57:23	global
rol_consultor	%	ROL	Y	caching_sha2_password	2026-10-06 00:57:23	global
rol_operador	%	ROL	Y	caching_sha2_password	2026-10-06 00:57:23	global

Bloque 2. Asignación rol → usuario

usuario	rol_asignado	puede_reasignar_rol	activo_por_defecto
admin_bd@localhost	rol_admin@%	N	SI
analista@localhost	rol_consultor@%	N	SI
operador@localhost	rol_operador@%	N	SI

Bloque 3. Privilegios de esquema

cuenta	esquema	privilegios	grant_option
'rol_admin'@'%'	Guia_3	ALTER, ALTER ROUTINE, CREATE, CREATE ROUTINE, CREATE TEMPORARY TABLES, CREATE VIEW, DELETE, DROP, EVENT, EXECUTE, INDEX, INSERT, LOCK TABLES, REFERENCES, SELECT, SHOW VIEW, TRIGGER, UPDATE	YES

Bloque 4. Privilegios por tabla

cuenta	tabla	privilegios	otorgado_por
auditor@localhost	auditoria_log	Select	root@localhost
rol_consultor@%	Estudiante	Select	root@localhost
rol_consultor@%	Grupo	Select	root@localhost
rol_consultor@%	Materia	Select	root@localhost
rol_consultor@%	Matricula	Select	root@localhost
rol_consultor@%	Profesor	Select	root@localhost
rol_consultor@%	Programa	Select	root@localhost
rol_operador@%	Estudiante	Select,Insert,Update	root@localhost
rol_operador@%	Grupo	Select,Insert,Update	root@localhost
rol_operador@%	Materia	Select,Insert,Update	root@localhost
rol_operador@%	Matricula	Select,Insert,Update	root@localhost
rol_operador@%	Profesor	Select,Insert,Update	root@localhost
rol_operador@%	Programa	Select,Insert,Update	root@localhost

Bloque 4. Vista pivote por tabla

tabla	rol_consultor	rol_operador	auditor
Estudiante	Select	Select,Insert,Update	-
Grupo	Select	Select,Insert,Update	-
Materia	Select	Select,Insert,Update	-
Matricula	Select	Select,Insert,Update	-
Profesor	Select	Select,Insert,Update	-
Programa	Select	Select,Insert,Update	-
auditoria_log	-	-	Select

Bloque 5. SHOW GRANTS auditor (directo)

Grants for auditor@localhost
GRANT USAGE ON *.* TO `auditor`@`localhost`
GRANT SELECT ON `Guia_3`.`auditoria_log` TO `auditor`@`localhost`

Bloque 5. SHOW GRANTS analista (membresía)

Grants for analista@localhost
GRANT USAGE ON *.* TO `analista`@`localhost`
GRANT `rol_consultor`@`%` TO `analista`@`localhost`

Bloque 5. SHOW GRANTS analista USING rol_consultor

Grants for analista@localhost
GRANT USAGE ON *.* TO `analista`@`localhost`
GRANT SELECT ON `Guia_3`.`Estudiante` TO `analista`@`localhost`
GRANT SELECT ON `Guia_3`.`Grupo` TO `analista`@`localhost`
GRANT SELECT ON `Guia_3`.`Materia` TO `analista`@`localhost`
GRANT SELECT ON `Guia_3`.`Matricula` TO `analista`@`localhost`
GRANT SELECT ON `Guia_3`.`Profesor` TO `analista`@`localhost`
GRANT SELECT ON `Guia_3`.`Programa` TO `analista`@`localhost`
GRANT `rol_consultor`@`%` TO `analista`@`localhost`

Bloque 5. SHOW GRANTS operador USING rol_operador

Grants for operador@localhost
GRANT USAGE ON *.* TO `operador`@`localhost`
GRANT SELECT, INSERT, UPDATE ON `Guia_3`.`Estudiante` TO `operador`@`localhost`
GRANT SELECT, INSERT, UPDATE ON `Guia_3`.`Grupo` TO `operador`@`localhost`
GRANT SELECT, INSERT, UPDATE ON `Guia_3`.`Materia` TO `operador`@`localhost`
GRANT SELECT, INSERT, UPDATE ON `Guia_3`.`Matricula` TO `operador`@`localhost`
GRANT SELECT, INSERT, UPDATE ON `Guia_3`.`Profesor` TO `operador`@`localhost`
GRANT SELECT, INSERT, UPDATE ON `Guia_3`.`Programa` TO `operador`@`localhost`
GRANT `rol_operador`@`%` TO `operador`@`localhost`

Bloque 5. SHOW GRANTS admin_bd USING rol_admin

Grants for admin_bd@localhost
GRANT USAGE ON *.* TO `admin_bd`@`localhost`
GRANT ALL PRIVILEGES ON `Guia_3`.* TO `admin_bd`@`localhost` WITH GRANT OPTION
GRANT `rol_admin`@`%` TO `admin_bd`@`localhost`

Bloque 6. Cuentas con GRANT OPTION

nivel	cuenta	objeto
global	debian-sys-maint@localhost	*.*
global	root@localhost	*.*
esquema	rol_admin@%	Guia_3.*

Bloque 7. Triggers instalados

trigger_nombre	momento	evento	tabla_vigilada	se_ejecuta_como	creado
trg_auditoria_estudiante_update	AFTER	UPDATE	Estudiante	root@localhost	2026-10-06 00:57:55.62
trg_auditoria_estudiante_delete	AFTER	DELETE	Estudiante	root@localhost	2026-10-06 00:57:55.62

Bloque 8. Resumen del log

tabla	operacion	usuario	eventos	primer_evento	ultimo_evento
Estudiante	UPDATE	operador@localhost	1	2026-10-06 00:57:55	2026-10-06 00:57:55
Estudiante	DELETE	admin_bd@localhost	1	2026-10-06 00:57:55	2026-10-06 00:57:55

Bloque 8. Detalle del log

id	tabla	operacion	usuario	fecha	id_afectado	telefono_anterior	direccion_anterior
1	Estudiante	UPDATE	operador@localhost	2026-10-06 00:57:55	1099887766	3201122334	Calle 100 #15-20
2	Estudiante	DELETE	admin_bd@localhost	2026-10-06 00:57:55	1099887766	3109988776	Carrera 7 #72-41

Bloque 9.1. Cuentas anónimas

cuentas_anonimas
0

Bloque 9.3. Cuentas desbloqueadas sin contraseña

User	Host	plugin
root	localhost	auth_socket

Bloque 9.4. Variables de seguridad

Variable_name	Value
activate_all_roles_on_login	OFF
bind_address	127.0.0.1
default_password_lifetime	0
have_ssl	YES
local_infile	OFF
log_bin	ON
mandatory_roles	
password_history	0
password_reuse_interval	0
require_secure_transport	OFF
tls_version	TLSv1.2,TLSv1.3
version	8.0.46-0ubuntu0.24.04.4

Nota. El bloque 9.2 no devolvió filas (ninguna cuenta de usuario desbloqueada acepta conexiones desde %) y el bloque 9.5 tampoco (no hay componentes instalados, por lo que `validate_password` no está activo; ver H7). Las cuentas `root` y `debian-sys-maint` son propias del servidor de prueba (Ubuntu), donde `root` se autentica por socket del sistema operativo.

Anexo C

Triggers de auditoría corregidos

Versión final de la actividad 4: usa `USER()` en lugar de `CURRENT_USER()` (hallazgo H3) y guarda la fila completa en `dato_anterior`.

```
-- ============================================================================
-- Practica 4 - Actividad 4: Triggers de auditoria (version corregida)
-- Sistema: Guia_3 | SGBD: MySQL 8.0
-- Cambios frente a la version inicial:
--   * USER() en lugar de CURRENT_USER(): dentro de un trigger CURRENT_USER()
--     devuelve el DEFINER (root) y no la cuenta que hizo el cambio (hallazgo H3).
--   * El JSON guarda la fila completa (se agregan PrefijoTelefono y FechaNacimiento).
-- Ejecutar como root.
-- ============================================================================
USE Guia_3;

CREATE TABLE IF NOT EXISTS auditoria_log (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    tabla         VARCHAR(64)              NOT NULL,
    operacion     ENUM('UPDATE', 'DELETE') NOT NULL,
    usuario       VARCHAR(100)             NOT NULL,
    fecha         DATETIME                 NOT NULL DEFAULT CURRENT_TIMESTAMP,
    dato_anterior JSON                     NULL
);

-- ----------------------------------------------------------------------------
-- AFTER UPDATE
-- ----------------------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_auditoria_estudiante_update;
DELIMITER $$
CREATE TRIGGER trg_auditoria_estudiante_update
AFTER UPDATE ON Estudiante
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_log (tabla, operacion, usuario, fecha, dato_anterior)
    VALUES (
        'Estudiante', 'UPDATE', USER(), NOW(),
        JSON_OBJECT(
            'IdEstudiante',     OLD.IdEstudiante,
            'PrimerNombre',     OLD.PrimerNombre,
            'SegundoNombre',    OLD.SegundoNombre,
            'PrimerApellido',   OLD.PrimerApellido,
            'SegundoApellido',  OLD.SegundoApellido,
            'CorreoEstudiante', OLD.CorreoEstudiante,
            'PrefijoTelefono',  OLD.PrefijoTelefono,
            'Telefono',         OLD.Telefono,
            'FechaNacimiento',  OLD.FechaNacimiento,
            'Direccion',        OLD.Direccion
        )
    );
END$$
DELIMITER ;

-- ----------------------------------------------------------------------------
-- AFTER DELETE
```

```
-- ------------------------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_auditoria_estudiante_delete;
DELIMITER $$
CREATE TRIGGER trg_auditoria_estudiante_delete
AFTER DELETE ON Estudiante
FOR EACH ROW
BEGIN
    INSERT INTO auditoria_log (tabla, operacion, usuario, fecha, dato_anterior)
    VALUES (
        'Estudiante', 'DELETE', USER(), NOW(),
        JSON_OBJECT(
            'IdEstudiante',     OLD.IdEstudiante,
            'PrimerNombre',     OLD.PrimerNombre,
            'SegundoNombre',    OLD.SegundoNombre,
            'PrimerApellido',   OLD.PrimerApellido,
            'SegundoApellido',  OLD.SegundoApellido,
            'CorreoEstudiante', OLD.CorreoEstudiante,
            'PrefijoTelefono',  OLD.PrefijoTelefono,
            'Telefono',         OLD.Telefono,
            'FechaNacimiento',  OLD.FechaNacimiento,
            'Direccion',        OLD.Direccion
        )
    );
END$$
DELIMITER ;

-- ------------------------------------------------------------------------------
-- Prueba (cada bloque en la sesion indicada)
-- ------------------------------------------------------------------------------
-- [root] Registro de prueba
-- INSERT INTO Estudiante VALUES (1099887766, 'Laura', 'Camila', 'Gomez', 'Rojas',
--   'laura.gomez@academia.umb.edu.co', '+57', '3201122334', '2004-06-20', 'Calle 100 #15-
20');
-- [operador] UPDATE  -> dispara trg_auditoria_estudiante_update
-- UPDATE Estudiante SET Telefono = '3109988776', Direccion = 'Carrera 7 #72-41'
--  WHERE IdEstudiante = 1099887766;
-- [admin_bd] DELETE  -> dispara trg_auditoria_estudiante_delete
-- DELETE FROM Estudiante WHERE IdEstudiante = 1099887766;
-- [auditor] Revision del rastro
-- SELECT id, tabla, operacion, usuario, fecha, dato_anterior FROM auditoria_log;
```

Anexo D

Validación de entradas en la aplicación

Fragmento de `Segura.py` (actividad 3) con la validación por lista blanca agregada antes de la consulta parametrizada (hallazgo H11, recomendación R15).

```
def es_documento_valido(valor):
    """Lista blanca: solo digitos, entre 6 y 10 caracteres (H11 / R15)."""
    return valor.isdigit() and 6 <= len(valor) <= 10

# Capa 1: validacion de entrada (rechaza antes de llegar a la BD)
if not es_documento_valido(payload_1):
    print("-> RECHAZADO por validacion: el documento debe ser numerico (6-10 digitos).")

# Capa 2: consulta parametrizada
query_segura_1 = ("SELECT IdEstudiante, PrimerNombre, PrimerApellido "
                  "FROM Estudiante WHERE IdEstudiante = %s")
cursor.execute(query_segura_1, (payload_1,))
```